

策略 2: 假设剪切片段的第一个数字为 a , 最后一个数字为 b , 要么把这个片段粘贴到 $a-1$ 的下一个位置, 要么粘贴到 $b+1$ 的前一个位置。

策略 3: 永远不要“破坏”一个已经连续排列的数字片段。例如, 不能把 1 2 3 4 中的 2 3 剪切出来。

3 种策略都能缩小状态空间, 但它们并不都是正确的。很多程序都无法得到“5 4 3 2 1”的正确结果(答案是 3 步而不是 4 步: $5\ 4\ 3\ 2\ 1 \rightarrow 3\ 2\ 5\ 4\ 1 \rightarrow 3\ 4\ 1\ 2\ 5 \rightarrow 1\ 2\ 3\ 4\ 5$), 读者不妨自行验证上面的 3 种策略是否可以得到这组数据的答案。

本题可以用 IDA* 算法求解。不难发现 $n \leq 9$ 时最多只需要 8 步, 因此深度上限为 8。IDA* 的关键在于启发函数。考虑后继不正确的数字个数 h , 可以证明每次剪切时 h 最多减少 3, 因此当 $3d+h > 3\max d$ 时可以剪枝, 其中 d 为当前深度, $\max d$ 为深度限制^①。

如何证明每次剪切时 h 最多减少 3 呢? 如图 7-19 所示, 因为最多只有 3 个数字的后继数字发生了改变(即图中的 a, b, c), h 自然最多减少 3。



图 7-19 h 最多减少 3

7.7 竞赛题目选讲

本章的篇幅不少, 但实际上介绍的算法很有系统性, 并不杂乱。这里先把这些算法和常见解决问题的思路总结一下, 然后选讲一些例题,

直接枚举。例如, 类似“1~ n 的整数中有多少个满足……”, “输入一个长度为 n 的序列, 有多少个连续子序列满足……”的问题都可以用直接枚举法。枚举法可以解决问题, 但是效率不一定足够高。第 8 章中将详细讨论算法效率的分析方法。

枚举子集和排列。 n 个元素的子集有 2^n 个, 可以用递归的方法枚举(前面介绍的增量法和位向量法都属于递归枚举), 也可以用二进制的方法枚举。递归法的优点在于效率高, 方便剪枝, 缺点在于代码比较长。一般来说, 当 n 很小(如 $n \leq 15$) 时, 会使用二进制的方式枚举。

n 个不同元素的全排列有 $n!$ 个。除了用递归的方法枚举之外, 还可以用 STL 的 `next_permutation` 来枚举, 它也适用于有重复元素的情形。

回溯法。简单地说, 回溯法几乎就是递归枚举, 只是多了一条: 违反题目要求时及时终止当前递归过程, 即回溯(backtracking)。回溯法最经典的题目就是八皇后问题, 这个问题也常常被作为“判断有没有学过回溯法”的依据。7.4 节的几个例题非常经典, 覆盖了回溯法的几个常见话题: 搜索对象的选取(天平难题)、最优性剪枝(带宽), 以及减少

^① 此处故意没有用前面介绍的 $h(s)$ 、 $g(s)$ 等记号。事实上, 经常采用这种直观的方式来思考, 而不去理会那些记号。

无用功(困难的串)。

状态空间搜索。从本质上讲,状态空间搜索算法和图算法的相似度比较大,但是图往往是“隐式”给出,所以这些算法又称“隐式图搜索”或者“产生式系统”^①。如果仔细品味前面《八数码问题》的解法,可以发现这个解法其实就是一个普通的 BFS 加上了“结点查找表”。前面介绍了 3 种方法实现结点查找表,各有用武之地。建议读者先熟练掌握后面两种(哈希表和 STL 集合),待学习完第 10 章后再尝试使用第一种方法(一一映射,或称“完美哈希”)。这些方法不仅能加快状态空间搜索的速度,还能给其他算法加速。第 8 章和第 9 章中将继续讨论这个问题。另外,双向广度优先搜索和 A* 等算法也有各自的用武之地,虽然限于篇幅未加介绍,但是笔者鼓励大家花一些时间搜索相关资料,并加以学习。例题中的“万圣节后的早晨”就是一处很好的“试验田”。

迭代加深搜索。本章最后介绍了迭代加深搜索。这是一个长期以来被“低估”了的算法,可以用来解决很多看起来更适合用 BFS 或者回溯法解决的问题,埃及分数问题就是一个绝好的例子,而例题“编辑书稿”也非常经典。

例题 7-11 宝箱 (Zombie's Treasure Chest, Shanghai 2011, UVa12325)

你有一个体积为 N 的箱子和两种数量无限的宝物。宝物 1 的体积为 S_1 , 价值为 V_1 ; 宝物 2 的体积为 S_2 , 价值为 V_2 。输入均为 32 位带符号整数。你的任务是计算最多能装多大价值的宝物。例如, $n=100$, $S_1=V_1=34$, $S_2=5$, $V_2=3$, 答案为 86, 方案是装两个宝物 1, 再装 6 个宝物 2。每种宝物都必须拿非负整数个。

【分析】

最容易想到的方法是:枚举宝物 1 的个数,然后尽量多拿宝物 2。这样做的时间复杂度为 $O(N/S_1)$, 当 N 和 S_1 相差非常悬殊时效率很低。当然,如果 N/S_2 很小时可以改成枚举宝物 2 的个数,所以这个方法不奏效的条件是: S_1 和 S_2 都很小,而 N 很大。

幸运的是, S_1 和 S_2 都很小时,有另外一种枚举法 B: S_2 个宝物 1 和 S_1 个宝物 2 的体积相等,而价值分别为 $S_2 \cdot V_1$ 和 $S_1 \cdot V_2$ 。如果前者比较大,则宝物 2 最多只会拿 S_1-1 个(否则可以把 S_1 个宝物 2 换成 S_2 个宝物 1); 如果后者比较大,则宝物 1 最多只会拿 S_2-1 个。不管是哪种情况,枚举量都只有 S_1 或者 S_2 。

这样,就得到了一个比较“另类”的分类枚举算法:

当 N/S_1 比较小时枚举宝物 1 的个数,时间复杂度为 $O(N/S_1)$, 否则,当 N/S_2 比较小时枚举宝物 2 的个数,时间复杂度为 $O(N/S_2)$, 否则说明 S_1 和 S_2 都比较小,执行枚举法 B,时间复杂度为 $O(\max\{S_1, S_2\})$ 。

例题 7-12 旋转游戏 (The Rotation Game, Shanghai 2004, UVa1343)

如图 7-20 所示形状的棋盘上分别有 8 个 1、2、3, 要往 A~H 方向旋转棋盘,使中间 8 个方格数字相同。图 7-20 (a) 进行 A 操作后变为图 7-20 (b), 再进行 C 操作后变为图 7-20 (c), 这正是一个目标状态(因为中间 8 个方格数字相同)。要求旋转次数最少。如果有多解,操作序列的字典序应尽量小。

^① 这个术语多用在传统人工智能书籍中,虽有一些描述上的差别,但本质相同。

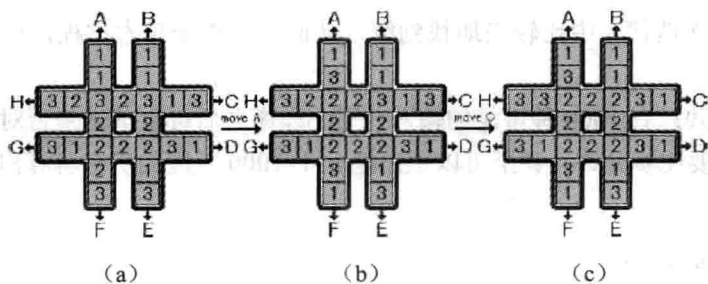


图 7-20 旋转游戏示意图

【分析】

本题是一个典型的状态空间搜索问题，可惜如果直接套用八数码问题的框架会超时。为什么？学完第 10 章的组合计数部分后会知道：8 个 1、8 个 2、8 个 3 的全排列个数为 $24!/(8!*8!*8!)=9465511770$ 。换句话说，最坏情况下最多要处理这么多结点！

解决方法很巧妙：本题要求的是中间 8 个数字相同，即 8 个 1 或者 8 个 2 或者 8 个 3。因此可以分 3 次求解。当目标是“中间 8 个数字都是 1”时，2 和 3 就没有区别了（都是“非 1”），因此状态总数变成了 8 个 1，16 个“非 1”的全排列个数，即 $24!/(8!*16!)=735471$ ，在可以接受的范围内了^①。

例题 7-13 快速幂计算 (Power Calculus, ACM/ICPC Yokohama 2006, UVa1374)

输入正整数 n ($1 \leq n \leq 1000$)，问最少需要几次乘除法可以从 x 得到 x^n ？例如， x^{31} 需要 6 次： $x^2=x*x$ ， $x^4=x^2*x^2$ ， $x^8=x^4*x^4$ ， $x^{16}=x^8*x^8$ ， $x^{32}=x^{16}*x^{16}$ ， $x^{31}=x^{32}/x$ 。计算过程中 x 的指数应当总是正整数（如 $x^{-3}=x/x^4$ 是不允许的）。

【分析】

这个题有一点“埃及分数”的味道，可以考虑迭代加深搜索。当前状态是已经得到的指数集合，操作是任选两个数进行加法和减法，并且不能产生重复的数，如图 7-21 所示。

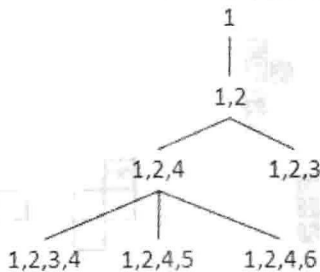


图 7-21 快速幂计算示意图

沿用之前的符号， d 表示当前深度， $\max d$ 表示深度上限，则如果当前序列最大的数乘以 $2^{\max d-d}$ 之后仍小于 n ，则剪枝（想一想，为什么）。另外，为了尽快接近目标，不应该“任选”两个数，而应该先选较大的数，并且先试加法再试减法^②。这样做可以在最后一次迭代

^① 一般来说，状态总数不超过 10^6 时都在可接受范围内。不过这只是一般规律，还要具体问题具体分析。

^② 这种技巧称为结点排序 (node ordering)。

（即找到解的那次迭代）中比较快地找到解，从而终止整个搜索过程，而不需要等整个解答树扩展完毕。

因为题目一共只有 1000 种可能的输入，写完程序之后可以试试是否对所有输入都能足够快地出解。只要比赛允许，甚至可以预先把 $n=1\sim 1000$ 范围的所有解算出来，输出成如下源代码：

```
#include<cstdio>
int answer[] = {0, 0, 1, ...}; //answer[1]=0, answer[2]=1, ...
int main() {
    int n;
    while(scanf("%d", &n) == 1 && n) printf("%d\n", answer[n]);
    return 0;
}
```

这样的技巧俗称“打表”。本题还有一些常见的优化，例如，限制减法的次数（实际上大部分时候都是最大的数乘以 2），或者限制超过 n 的数的个数（事实上，可以证明最多有一个数需要超过 n ），读者不妨一试。

例题 7-14 网格动物 (Lattice Animals, ACM/ICPC NEERC 2004, UVa1602)

输入 n, w, h ($1 \leq n \leq 10, 1 \leq w, h \leq n$)，求能放在 $w \times h$ 网格里的不同的 n 连块的个数（注意，平移、旋转、翻转后相同的算作同一种）。例如， 2×4 里的 5 连块有 5 种（第一行），而 3×3 里的 8 连块有以下 3 种（第二行），如图 7-22 所示。

【分析】

本题看上去没有什么好办法，只能用回溯法求解。如何求解呢？首先需要确定搜索对象。因为要求各个格子连通，所以可以把“连通块”作为搜索对象，每次枚举一个位置，然后放一个新的块，最后判重，如图 7-23 所示。

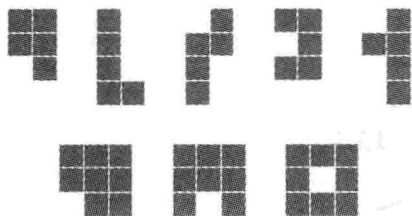


图 7-22 网格动物例题示意图

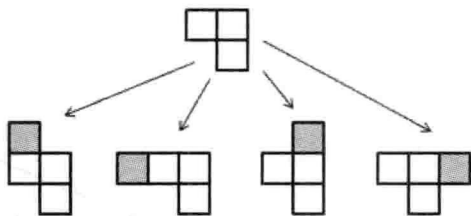


图 7-23 回溯法求解

需要注意的是，如果采用最简单的写法，每个 n 连块都会被重复枚举很多次（想一想，为什么）。也可以用前面介绍过的方法判重，但实际上有办法确保每个 n 连块恰好被枚举一次，由 Redelmeier 发现，有兴趣的读者可以自行研究^①。

本题非常经典，强烈建议读者编写程序。

^① 可以参考 en.wikipedia.org/wiki/Polyomino。